Playwright setup / installation

Prerequisites
--> Node.js (latest LTS version) installed
    check with: node -v   and   npm -v
--> VS Code (recommended editor)

Step 1: Create a project folder
--> Create a folder, e.g. PlaywrightDemo, and open it in VS Code

Step 2: Initialize Playwright (run in the VS Code terminal)
--> npm init playwright@latest

Step 3: Answer the prompts
--> Language   : TypeScript or JavaScript
--> Tests folder name : tests
--> Add a GitHub Actions workflow : optional (No for learning)
--> Install Playwright browsers   : Yes
    (downloads Chromium, Firefox and WebKit)

PLAYWRIGHT PROJECT STRUCTURE

1) .github
 -> Contains GitHub-related configuration files used to manage and automate
    the project, especially CI/CD workflows.
 -> The playwright.yml file automatically runs Playwright tests when code is
    pushed to GitHub (or a pull request is opened).

2) node_modules
 -> Contains all the packages and dependencies required for the Playwright project.
 -> When we run "npm install" or "npm init playwright@latest", the required
    packages are downloaded and stored inside the node_modules folder.

3) tests
 -> Contains the actual test scripts (e.g. example.spec.ts).

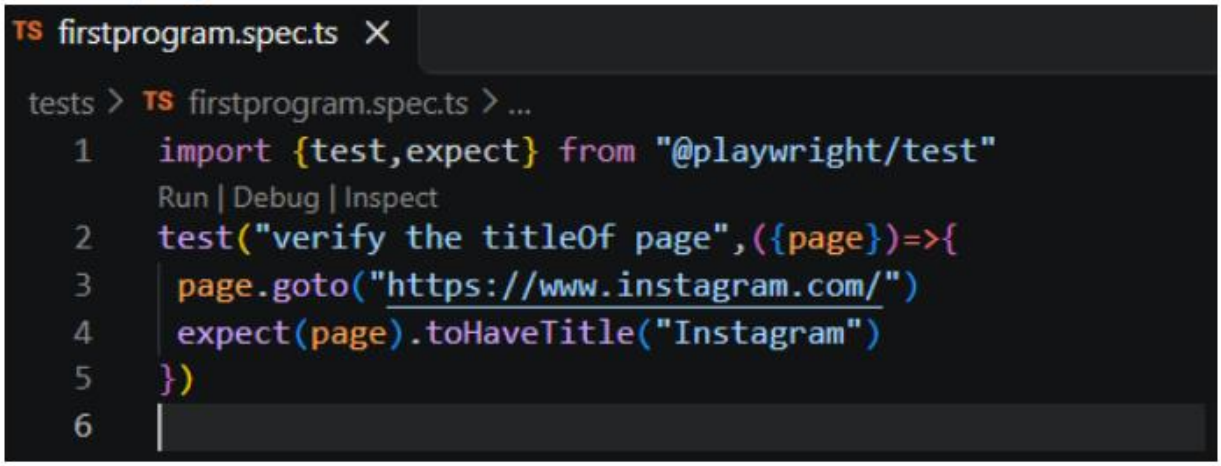
4) .gitignore
 -> Tells Git which files and folders should not be tracked or uploaded to GitHub (e.g. node_modules, test-results, playwright-report).

5) package-lock.json
 -> Maintains the exact versions of the installed npm dependencies to provide consistent installation on every machine.

6) package.json
 -> Contains project info, scripts, and the required dependencies for a Node.js/Playwright project.

7) playwright.config.ts
 -> The main configuration file of the Playwright project. It can contain: browser settings (projects: Chromium, Firefox, WebKit), testDir, base URL, timeout, retries, workers, screenshots, videos, traces, and reporter.

first program



Problems in this program

-->Whenever we create a step inside the test, all the steps will return a promise.
-->Now we should understand what is promise, await and async.
-->There are 2 things: Synchronous and Asynchronous.

What is synchronous and asynchronous in nature?

--> JavaScript and TypeScript code will follow both sync and async nature.

Synchronous
step 1
step 2
step 3

{First step will execute, and once it is completed step 2 will execute, and so on. So this is synchronous nature }

-->Java and Python will follow sync nature.

Asynchronous
step 1 -->
step 2 -->
step 3 -->

{It is executing some task in the background. Before completing this task,step 2 will start executing.Same with step 3.}

-->So this is asynchronous nature.
--> In TypeScript and JavaScript, certain statements will follow async nature.

So,
--> Sync means step by step execution
--> Async means the next step does not wait for the previous step to complete
--> Because of this async, our script might fail.
So to solve this problem...

How can we solve it?
1) We need to wait until step 1 is completed.
2) Till then, step 2 should wait until the first job is completed.
3) The first job (step 1) may complete or may not complete,
   may be successful or may not be successful.
4) So we need a confirmation whether the task is completed or not.
   This confirmation is called a promise.
5) Promise has 3 states:
   Pending : the task is still running
   Resolved : the task is successful
   Rejected : the task is unsuccessful
6) To wait until the task or execution is completed, we have to use await,
   for the step which performs a task and returns a promise.
   So here, await will wait until the step is completed.
7) A function that uses await must be marked async.

